

Netflix Interview Preparation — Database & SQL Roles

1. Company Profile

Engineering Culture

- **Freedom and Responsibility:** high autonomy, high accountability — no micromanagement
- "Highly aligned, loosely coupled" — teams own their data end-to-end
- Open source first: Conductor, Hollow, Hystrix, Eureka, Zuul all originated at Netflix
- Chaos Engineering culture: systems must survive failures gracefully (Chaos Monkey)
- Data-driven content decisions: every greenlighting decision backed by user data
- 247M subscribers, 190+ countries — scale is the baseline assumption

Database Technology Stack

Layer	Technology
Structured Data	MySQL (Vitess for sharding)
Wide Column	Apache Cassandra (primary store for viewing history, profiles)
In-Memory Cache	EVCache (Netflix's Memcache-based distributed cache)
PostgreSQL	Internal tools, billing, content metadata
Analytics DW	Apache Iceberg on S3, Spark, Druid
OLAP	Druid (real-time analytics), Presto
Graph	Custom graph store for content relationships
Search	Elasticsearch
Stream Processing	Apache Flink, Apache Kafka
Feature Store	Custom ML feature store (Feast-like)

What Netflix Values in DB/SQL Candidates

- Large-scale data pipeline design (billions of events/day from streaming)
 - Analytical SQL on distributed systems (Presto, Spark SQL, Hive)
 - Content/recommendation system schema design
 - A/B testing infrastructure and analysis
 - Data reliability: pipelines that self-heal, data quality enforcement
 - Ownership mentality — freedom and responsibility is real
-

2. Interview Process

Typical Rounds (Senior SWE / Data Engineer / MLE)

Round	Format	Duration	Focus
-------	--------	----------	-------

Recruiter Screen	Phone	30 min	Background, culture fit
Technical Screen	CoderPad/Hackerrank	60–90 min	SQL + data pipeline problems
Onsite 1	CoderPad	60 min	SQL complex analysis
Onsite 2	Video/Whiteboard	60 min	System design (data)
Onsite 3	Video	60 min	Data architecture
Onsite 4	Video	45 min	Behavioral (Freedom & Responsibility)
Onsite 5	Video	45 min	Domain deep dive (ML/streaming/etc.)

Level Expectations

Level	SQL	Data Architecture	Autonomy Signal
L4	Hard queries, streaming context	Service-level designs	Can work independently
L5	Expert SQL + distributed	Full platform designs	Has built systems independently
L6	Novel approaches	Org-level data strategy	Has influenced platform direction

3. SQL Coding Round — 25 Questions

Q1. Find the top 10 most-watched shows by total watch time in the last 30 days.

```
SELECT
  s.show_id,
  s.title,
  SUM(v.watch_duration_min) AS total_watch_min,
  COUNT(DISTINCT v.user_id) AS unique_viewers,
  ROUND(SUM(v.watch_duration_min) / 60.0, 0) AS total_watch_hours
FROM viewing_history v
JOIN shows s ON v.content_id = s.show_id AND v.content_type = 'show'
WHERE v.watched_at >= NOW() - INTERVAL '30 days'
GROUP BY s.show_id, s.title
ORDER BY total_watch_min DESC
LIMIT 10;
```

Q2. Calculate completion rate per show (users who watched > 90% of all episodes).

```
WITH user_show_progress AS (
  SELECT
    v.user_id,
    e.show_id,
    COUNT(DISTINCT e.episode_id) AS episodes_watched,
    (SELECT COUNT(*) FROM episodes WHERE show_id = e.show_id) AS total_episodes
  FROM viewing_history v
```

```

        JOIN episodes e ON v.content_id = e.episode_id AND v.content_type = 'episode'
        WHERE v.completion_pct >= 90
        GROUP BY v.user_id, e.show_id
    )
    SELECT
        s.title,
        COUNT(DISTINCT usp.user_id) FILTER (WHERE usp.episodes_watched >=
usp.total_episodes * 0.9) AS completed_users,
        COUNT(DISTINCT usp.user_id) AS started_users,
        ROUND(100.0 * COUNT(DISTINCT usp.user_id) FILTER (WHERE usp.episodes_watched >=
usp.total_episodes * 0.9)
            / NULLIF(COUNT(DISTINCT usp.user_id), 0), 2) AS completion_rate
    FROM user_show_progress usp
    JOIN shows s ON usp.show_id = s.show_id
    GROUP BY s.title
    ORDER BY completion_rate DESC;

```

Q3. Identify users who watch content in multiple genres simultaneously (genre diversity score).

```

    SELECT
        user_id,
        COUNT(DISTINCT c.genre) AS distinct_genres,
        ARRAY_AGG(DISTINCT c.genre ORDER BY c.genre) AS genres_watched,
        SUM(watch_duration_min) AS total_watch_min
    FROM viewing_history v
    JOIN content c ON v.content_id = c.content_id
    WHERE v.watched_at >= NOW() - INTERVAL '30 days'
    GROUP BY user_id
    ORDER BY distinct_genres DESC;

```

Q4. Find shows with a "binge-watching" pattern — users who watched 3+ episodes in one sitting.

```

    WITH viewing_sessions AS (
        SELECT
            user_id,
            content_id,
            watched_at,
            watched_at - LAG(watched_at) OVER (PARTITION BY user_id ORDER BY watched_at)
        AS gap
        FROM viewing_history
        WHERE content_type = 'episode'
    ),
    sessions_numbered AS (
        SELECT *,
            SUM(CASE WHEN gap IS NULL OR gap > INTERVAL '1 hour' THEN 1 ELSE 0 END)
            OVER (PARTITION BY user_id ORDER BY watched_at) AS session_id
        FROM viewing_sessions
    ),
    binge_sessions AS (
        SELECT
            user_id,

```

```

        session_id,
        e.show_id,
        COUNT(*) AS episodes_in_session
    FROM sessions_numbered sn
    JOIN episodes e ON sn.content_id = e.episode_id
    GROUP BY user_id, session_id, e.show_id
    HAVING COUNT(*) >= 3
)
SELECT
    s.title,
    COUNT(DISTINCT bs.user_id) AS binge_users,
    AVG(bs.episodes_in_session) AS avg_binge_episodes
FROM binge_sessions bs
JOIN shows s USING (show_id)
GROUP BY s.title
ORDER BY binge_users DESC;

```

Q5. A/B test analysis: compare mean watch time between control and treatment.

```

SELECT
    variant,
    COUNT(DISTINCT user_id) AS users,
    ROUND(AVG(daily_watch_min), 2) AS avg_daily_watch_min,
    ROUND(STDDEV(daily_watch_min), 2) AS stddev,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY daily_watch_min) AS
median_daily_watch
FROM (
    SELECT
        ab.user_id,
        ab.variant,
        SUM(v.watch_duration_min) / NULLIF(COUNT(DISTINCT DATE(v.watched_at)), 0) AS
daily_watch_min
    FROM ab_test_assignments ab
    LEFT JOIN viewing_history v ON ab.user_id = v.user_id
        AND v.watched_at >= ab.assigned_at
    WHERE ab.test_id = 'homepage_redesign_v2'
    GROUP BY ab.user_id, ab.variant
) user_stats
GROUP BY variant;

```

Q6. Content recommendation quality: track whether users actually watch recommended content.

```

SELECT
    r.algorithm_version,
    COUNT(*) AS total_recommendations,
    COUNT(*) FILTER (WHERE v.content_id IS NOT NULL) AS watched,
    ROUND(100.0 * COUNT(*) FILTER (WHERE v.content_id IS NOT NULL) / COUNT(*), 2) AS
watch_rate,
    ROUND(AVG(v.completion_pct) FILTER (WHERE v.completion_id IS NOT NULL), 2) AS
avg_completion_when_watched
FROM recommendations r

```

```

LEFT JOIN viewing_history v ON r.user_id = v.user_id
                           AND r.content_id = v.content_id
                           AND v.watched_at BETWEEN r.shown_at AND r.shown_at +
INTERVAL '48 hours'
GROUP BY r.algorithm_version
ORDER BY watch_rate DESC;

```

Q7. Find users who started a series but never came back after episode 1 (drop-off analysis).

```

WITH series_starters AS (
    SELECT DISTINCT v.user_id, e.show_id
    FROM viewing_history v
    JOIN episodes e ON v.content_id = e.episode_id
    WHERE e.season_number = 1 AND e.episode_number = 1
),
continued AS (
    SELECT DISTINCT v.user_id, e.show_id
    FROM viewing_history v
    JOIN episodes e ON v.content_id = e.episode_id
    WHERE NOT (e.season_number = 1 AND e.episode_number = 1)
)
SELECT
    ss.show_id,
    s.title,
    COUNT(DISTINCT ss.user_id) AS started,
    COUNT(DISTINCT c.user_id) AS continued,
    COUNT(DISTINCT ss.user_id) - COUNT(DISTINCT c.user_id) AS dropped_after_ep1,
    ROUND(100.0 * (COUNT(DISTINCT ss.user_id) - COUNT(DISTINCT c.user_id))
          / NULLIF(COUNT(DISTINCT ss.user_id), 0), 2) AS drop_rate_pct
FROM series_starters ss
LEFT JOIN continued c ON ss.user_id = c.user_id AND ss.show_id = c.show_id
JOIN shows s ON ss.show_id = s.show_id
GROUP BY ss.show_id, s.title
ORDER BY drop_rate_pct DESC;

```

Q8. Streaming quality analysis: detect users experiencing frequent rebuffering events.

```

SELECT
    user_id,
    COUNT(*) AS total_playback_sessions,
    COUNT(*) FILTER (WHERE rebuffer_count > 2) AS degraded_sessions,
    AVG(rebuffer_ratio) AS avg_rebuffer_ratio,
    ROUND(100.0 * COUNT(*) FILTER (WHERE rebuffer_count > 2) / COUNT(*), 2) AS
degraded_pct
FROM playback_sessions
WHERE started_at >= NOW() - INTERVAL '7 days'
GROUP BY user_id
HAVING COUNT(*) >= 5 -- minimum sessions for meaningful analysis
ORDER BY degraded_pct DESC;

```

Q9. Show rating prediction: calculate Bayesian average to avoid small-sample bias.

```
WITH global_stats AS (
    SELECT AVG(rating) AS global_avg, COUNT(*) AS global_count FROM content_ratings
),
show_ratings AS (
    SELECT
        content_id,
        COUNT(*) AS review_count,
        AVG(rating) AS raw_avg_rating
    FROM content_ratings
    GROUP BY content_id
)
SELECT
    content_id,
    review_count,
    raw_avg_rating,
    -- Bayesian average: (C * m + n * R) / (C + n)
    -- where C=global avg, m=minimum reviews threshold (100), n=actual count, R=raw
    avg
    ROUND(
        (100 * gs.global_avg + sr.review_count * sr.raw_avg_rating) /
        NULLIF(100 + sr.review_count, 0),
        4
    ) AS bayesian_avg_rating
FROM show_ratings sr
CROSS JOIN global_stats gs
ORDER BY bayesian_avg_rating DESC;
```

Q10. Find content that performs significantly better in specific regions.

```
SELECT
    c.title,
    v.country_code,
    COUNT(DISTINCT v.user_id) AS viewers,
    SUM(v.watch_duration_min) AS total_watch_min,
    AVG(v.completion_pct) AS avg_completion,
    -- Z-score vs. global average
    (AVG(v.completion_pct) - AVG(AVG(v.completion_pct)) OVER ())
    / NULLIF(STDDEV(AVG(v.completion_pct)) OVER (), 0) AS completion_z_score
FROM viewing_history v
JOIN content c ON v.content_id = c.content_id
GROUP BY c.title, v.country_code
HAVING COUNT(DISTINCT v.user_id) > 1000
ORDER BY completion_z_score DESC;
```

Q11. Calculate "subscriber acquisition efficiency" — shows that drove the most new signups.

```
WITH new_subscribers AS (
    SELECT user_id, DATE(signup_date) AS signup_day
```

```

FROM users
WHERE signup_date >= NOW() - INTERVAL '90 days'
),
first_watch AS (
    SELECT DISTINCT ON (ns.user_id)
        ns.user_id,
        v.content_id,
        c.title,
        c.genre
    FROM new_subscribers ns
    JOIN viewing_history v ON ns.user_id = v.user_id
    ORDER BY ns.user_id, v.watched_at
)
SELECT
    title,
    genre,
    COUNT(*) AS new_subscribers_first_watched,
    ROUND(100.0 * COUNT(*) / (SELECT COUNT(*) FROM new_subscribers), 2) AS
pct_of_new_subs
FROM first_watch
GROUP BY title, genre
ORDER BY new_subscribers_first_watched DESC
LIMIT 20;

```

Q12. Churn prediction feature: calculate recency, frequency, monetary (RFM) for subscribers.

```

SELECT
    user_id,
    CURRENT_DATE - MAX(DATE(watched_at)) AS recency_days,
    COUNT(DISTINCT DATE(watched_at)) AS frequency_days,
    SUM(watch_duration_min) AS total_watch_min,
    -- RFM scoring
    NTILE(5) OVER (ORDER BY MAX(watched_at) DESC) AS r_score,
    NTILE(5) OVER (ORDER BY COUNT(DISTINCT DATE(watched_at)) DESC) AS f_score,
    NTILE(5) OVER (ORDER BY SUM(watch_duration_min) DESC) AS m_score
FROM viewing_history
WHERE watched_at >= NOW() - INTERVAL '90 days'
GROUP BY user_id;

```

Q13. Language preference analysis: detect users who switched their preferred language.

```

WITH weekly_language AS (
    SELECT
        user_id,
        DATE_TRUNC('week', watched_at) AS week,
        audio_language,
        COUNT(*) AS views,
        RANK() OVER (PARTITION BY user_id, DATE_TRUNC('week', watched_at) ORDER BY
COUNT(*) DESC) AS lang_rank
    FROM viewing_history
    GROUP BY user_id, DATE_TRUNC('week', watched_at), audio_language

```

```

),
primary_language AS (
    SELECT user_id, week, audio_language AS primary_lang
    FROM weekly_language WHERE lang_rank = 1
)
SELECT
    user_id,
    week,
    primary_lang,
    LAG(primary_lang) OVER (PARTITION BY user_id ORDER BY week) AS prev_primary_lang
FROM primary_language
WHERE primary_lang <> LAG(primary_lang) OVER (PARTITION BY user_id ORDER BY week);

```

Q14. Content gap analysis: genres with high search but low content supply.

```

WITH search_demand AS (
    SELECT genre_searched, COUNT(*) AS search_count
    FROM search_events
    WHERE searched_at >= NOW() - INTERVAL '30 days'
    GROUP BY genre_searched
),
content_supply AS (
    SELECT genre, COUNT(*) AS content_count
    FROM content
    WHERE is_available = TRUE
    GROUP BY genre
)
SELECT
    sd.genre_searched,
    sd.search_count,
    COALESCE(cs.content_count, 0) AS available_titles,
    ROUND(sd.search_count::numeric / NULLIF(COALESCE(cs.content_count, 0), 0), 2) AS
demand_per_title
FROM search_demand sd
LEFT JOIN content_supply cs ON sd.genre_searched = cs.genre
ORDER BY demand_per_title DESC NULLS FIRST;

```

Q15. Calculate subscriber churn probability signals.

```

SELECT
    u.user_id,
    u.plan_type,
    CURRENT_DATE - MAX(DATE(v.watched_at)) AS days_since_last_watch,
    COUNT(DISTINCT DATE(v.watched_at)) FILTER (WHERE v.watched_at >= NOW() -
INTERVAL '30 days') AS active_days_last_30,
    COUNT(DISTINCT DATE(v.watched_at)) FILTER (WHERE v.watched_at BETWEEN NOW() -
INTERVAL '90 days' AND NOW() - INTERVAL '31 days') AS active_days_prev_30_60,
    -- Decline signal
    COUNT(DISTINCT DATE(v.watched_at)) FILTER (WHERE v.watched_at >= NOW() -
INTERVAL '30 days') <
    COUNT(DISTINCT DATE(v.watched_at)) FILTER (WHERE v.watched_at BETWEEN NOW() -

```



```

INTERVAL '60 days' AND NOW() - INTERVAL '31 days') AS declining_activity
FROM users u
LEFT JOIN viewing_history v ON u.user_id = v.user_id
WHERE u.subscription_status = 'active'
GROUP BY u.user_id, u.plan_type
HAVING CURRENT_DATE - MAX(DATE(v.watched_at)) > 14
ORDER BY days_since_last_watch DESC;

```

Q16–Q25: Advanced Analytics Queries

Q16. Moving average of daily subscribers added/lost.

```

SELECT
    subscription_date,
    net_change,
    SUM(net_change) OVER (ORDER BY subscription_date ROWS BETWEEN 6 PRECEDING AND
CURRENT ROW) AS rolling_7d_sum,
    AVG(net_change) OVER (ORDER BY subscription_date ROWS BETWEEN 29 PRECEDING AND
CURRENT ROW) AS rolling_30d_avg
FROM (
    SELECT
        DATE(changed_at) AS subscription_date,
        SUM(CASE WHEN action = 'subscribe' THEN 1 ELSE -1 END) AS net_change
    FROM subscription_events
    GROUP BY DATE(changed_at)
) daily;

```

Q17. Content affinity matrix — which genres are most often watched together.

```

WITH user_genres AS (
    SELECT DISTINCT user_id, c.genre
    FROM viewing_history v
    JOIN content c ON v.content_id = c.content_id
    WHERE v.watched_at >= NOW() - INTERVAL '30 days'
)
SELECT
    a.genre AS genre_a,
    b.genre AS genre_b,
    COUNT(DISTINCT a.user_id) AS users_watch_both,
    COUNT(DISTINCT a.user_id)::numeric / (
        SELECT COUNT(DISTINCT user_id) FROM user_genres WHERE genre = a.genre
    ) AS co_watch_rate
FROM user_genres a
JOIN user_genres b ON a.user_id = b.user_id AND a.genre < b.genre
GROUP BY a.genre, b.genre
ORDER BY co_watch_rate DESC;

```

Q18. Multi-profile household analysis: detect distinct viewer personas within one account.

```

SELECT
    account_id,
    profile_id,
    COUNT(DISTINCT v.content_id) AS unique_titles_watched,
    ARRAY_AGG(DISTINCT c.genre ORDER BY c.genre) AS genres,
    AVG(v.watch_duration_min) AS avg_session_min,
    -- Peak viewing hours
    MODE() WITHIN GROUP (ORDER BY EXTRACT(HOUR FROM v.watched_at)) AS peak_hour
FROM viewing_history v
JOIN profiles p ON v.profile_id = p.profile_id
JOIN content c ON v.content_id = c.content_id
GROUP BY account_id, profile_id;

```

Q19. Engagement funnel for original content launch.

```

WITH launch_funnel AS (
    SELECT
        content_id,
        SUM(CASE WHEN event_type = 'thumbnail_impression' THEN 1 END) AS
impressions,
        SUM(CASE WHEN event_type = 'play_started' THEN 1 END) AS plays,
        SUM(CASE WHEN event_type = 'play_completed' THEN 1 END) AS completions,
        SUM(CASE WHEN event_type = 'rated' THEN 1 END) AS ratings,
        SUM(CASE WHEN event_type = 'shared' THEN 1 END) AS shares
    FROM content_events
    WHERE content_id IN (SELECT content_id FROM content WHERE release_date =
CURRENT_DATE - 7)
    GROUP BY content_id
)
SELECT
    c.title,
    lf.impressions,
    lf.plays,
    lf.completions,
    ROUND(100.0 * lf.plays / NULLIF(lf.impressions, 0), 2) AS click_rate,
    ROUND(100.0 * lf.completions / NULLIF(lf.plays, 0), 2) AS completion_rate,
    ROUND(100.0 * lf.ratings / NULLIF(lf.completions, 0), 2) AS rating_rate
FROM launch_funnel lf
JOIN content c USING (content_id);

```

Q20. International expansion analysis: identify markets with fastest engagement growth.

```

SELECT
    country_code,
    DATE_TRUNC('month', watched_at) AS month,
    COUNT(DISTINCT user_id) AS mau,
    SUM(watch_duration_min) AS total_watch_min,
    LAG(COUNT(DISTINCT user_id)) OVER (PARTITION BY country_code ORDER BY
DATE_TRUNC('month', watched_at)) AS prev_month_mau,
    ROUND(100.0 * (COUNT(DISTINCT user_id) - LAG(COUNT(DISTINCT user_id)) OVER

```

```
(PARTITION BY country_code ORDER BY DATE_TRUNC('month', watched_at)))
    / NULLIF(LAG(COUNT(DISTINCT user_id)) OVER (PARTITION BY country_code
ORDER BY DATE_TRUNC('month', watched_at)), 0), 2) AS mau_growth_pct
FROM viewing_history
GROUP BY country_code, DATE_TRUNC('month', watched_at)
ORDER BY country_code, month;
```

Q21-Q25 (Advanced):

- Q21: Cohort analysis — 90-day retention by signup month
- Q22: Personalization A/B test: which thumbnail drives highest CTR per genre
- Q23: Content licensing cost vs. revenue contribution per title
- Q24: Detect subtitle/audio sync issues via user behavior signals
- Q25: Predict next-best content to serve using collaborative filtering signals

4. Database Design Tasks

Task 1: Content Delivery and Metadata Schema

```
CREATE TABLE content (
    content_id      BIGSERIAL PRIMARY KEY,
    title           VARCHAR(500) NOT NULL,
    content_type    VARCHAR(20) CHECK (content_type IN
('movie', 'series', 'documentary', 'short', 'interactive')),
    genre           VARCHAR(100) [],
    release_date    DATE,
    maturity_rating VARCHAR(10),
    original_language CHAR(5),
    production_country CHAR(2),
    imdb_id         VARCHAR(20),
    synopsis        TEXT,
    search_vector   TSVECTOR GENERATED ALWAYS AS (
        TO_TSVECTOR('english', COALESCE(title, '') || ' ' || COALESCE(synopsis, ''))
    ) STORED
);

CREATE TABLE shows (
    show_id         BIGINT PRIMARY KEY REFERENCES content(content_id),
    total_seasons   INT,
    status          VARCHAR(20) CHECK (status IN
('ongoing', 'ended', 'cancelled', 'upcoming'))
);

CREATE TABLE episodes (
    episode_id      BIGSERIAL PRIMARY KEY,
    show_id         BIGINT REFERENCES shows(show_id),
    season_number   INT,
    episode_number  INT,
    title           VARCHAR(500),
    duration_min    INT,
    release_date    DATE,
```

```

    UNIQUE (show_id, season_number, episode_number)
);

CREATE TABLE content_availability (
    content_id      BIGINT NOT NULL,
    country_code    CHAR(2) NOT NULL,
    available_from  DATE NOT NULL,
    available_to    DATE,
    plan_restriction TEXT[], -- ['basic','standard','premium'] or empty = all plans
    PRIMARY KEY (content_id, country_code)
);

CREATE TABLE content_assets (
    asset_id        BIGSERIAL PRIMARY KEY,
    content_id      BIGINT,
    episode_id      BIGINT,
    quality         VARCHAR(10) CHECK (quality IN
('SD','HD','FHD','4K','HDR','Dolby')),
    language        CHAR(5),
    asset_type      VARCHAR(20) CHECK (asset_type IN
('video','subtitle','audio','thumbnail')),
    cdn_url         TEXT NOT NULL,
    file_size_mb    NUMERIC(10,2)
);

```

Task 2: A/B Testing Infrastructure Schema

```

CREATE TABLE experiments (
    experiment_id  BIGSERIAL PRIMARY KEY,
    name          VARCHAR(200) UNIQUE NOT NULL,
    hypothesis     TEXT,
    primary_metric VARCHAR(100),
    status        VARCHAR(20) DEFAULT 'draft',
    started_at    TIMESTAMPTZ,
    ended_at      TIMESTAMPTZ,
    created_by    BIGINT
);

CREATE TABLE experiment_variants (
    variant_id     BIGSERIAL PRIMARY KEY,
    experiment_id  BIGINT REFERENCES experiments(experiment_id),
    name          VARCHAR(100), -- 'control','treatment_a','treatment_b'
    traffic_pct    NUMERIC(5,2),
    config         JSONB -- variant-specific configuration
);

CREATE TABLE experiment_assignments (
    user_id        BIGINT NOT NULL,
    experiment_id  BIGINT NOT NULL,
    variant_id     BIGINT NOT NULL,
    assigned_at    TIMESTAMPTZ DEFAULT NOW(),
);

```

```

        PRIMARY KEY (user_id, experiment_id)
    ) PARTITION BY RANGE (assigned_at);

-- Experiment metrics (denormalized for fast analysis)
CREATE TABLE experiment_metrics (
    user_id          BIGINT NOT NULL,
    experiment_id    BIGINT NOT NULL,
    metric_name      VARCHAR(100),
    metric_value     NUMERIC(15,4),
    recorded_at      TIMESTAMPTZ DEFAULT NOW()
) PARTITION BY RANGE (recorded_at);

```

Task 3: Recommendation System Database Schema

```

-- User interaction signals
CREATE TABLE user_content_interactions (
    user_id          BIGINT NOT NULL,
    content_id       BIGINT NOT NULL,
    interaction_type  VARCHAR(30) CHECK (interaction_type IN
('view','complete','like','dislike','add_to_list','remove_from_list','search')),
    value            NUMERIC(6,4) DEFAULT 1.0, -- implicit feedback weight
    occurred_at      TIMESTAMPTZ DEFAULT NOW(),
    PRIMARY KEY (user_id, content_id, interaction_type, occurred_at)
) PARTITION BY RANGE (occurred_at);

-- Pre-computed similarity scores (batch updated daily)
CREATE TABLE content_similarity (
    content_a        BIGINT NOT NULL,
    content_b        BIGINT NOT NULL,
    similarity_score  NUMERIC(8,6) NOT NULL,
    algorithm        VARCHAR(30),
    computed_date    DATE NOT NULL DEFAULT CURRENT_DATE,
    PRIMARY KEY (content_a, content_b, computed_date)
);

-- User recommendation cache (updated per user session)
CREATE TABLE user_recommendations (
    user_id          BIGINT NOT NULL,
    shelf_type       VARCHAR(50), --
'continue_watching','trending','because_you_watched','top_picks'
    content_ids      BIGINT[] NOT NULL, -- ordered list
    generated_at     TIMESTAMPTZ DEFAULT NOW(),
    algorithm_version VARCHAR(20),
    PRIMARY KEY (user_id, shelf_type)
);

```

5. System Design Scenarios

Scenario 1: Design Netflix's Viewing History Service

Requirements: 247M users, 1B+ view events/day, fast retrieval for "Continue Watching"

Architecture:

Write Path:

Video Player → Event Service → Kafka (view_events topic)

→ Flink Consumer:

- (a) Update Redis: HSET user:{user_id}:progress content:{id} {seconds_watched}
- (b) Batch write to Cassandra (durable, partitioned by user_id)
- (c) Batch aggregate to Iceberg/S3 (analytics)

Read Path:

GET /continue-watching?user_id={id}

- Redis (< 1ms): get top 20 in-progress content
- If cache miss → Cassandra (< 5ms): query recent 30 days

Schema in Cassandra:

Table: viewing_history (user_id, content_id, profile_id, progress_s, last_watched)

Partition key: user_id

Clustering key: last_watched DESC (most recent first)

PostgreSQL role:

- Aggregate analytics (daily/weekly stats)
- A/B test analysis
- Content performance dashboards
- Billing and subscription data

Scenario 2: Design Personalized Homepage Rendering

Components: Pre-compute shelves (rows of content) for 247M users

Key DB decisions:

```
-- Shelf computation pipeline (offline, Apache Spark)
-- Result stored per user
CREATE TABLE homepage_config (
  user_id          BIGINT PRIMARY KEY,
  shelves          JSONB NOT NULL, -- ordered list of shelf configs with
content_id arrays
  generated_at     TIMESTAMPTZ DEFAULT NOW(),
  ttl_minutes      INT DEFAULT 60
);

-- Each shelf = {type, title, content_ids: [...], algorithm}
-- Redis: cache homepage for active users (TTL 30 min)
-- PostgreSQL: source of truth for analytics
```

Scenario 3: Design Streaming Quality Analytics

Requirements: Detect and respond to quality degradation affecting users in real-time.

Data Flow:

Player → 1 event per second (bitrate, buffer events, CDN node)
Kafka → Flink (5-min tumbling window aggregates per CDN node, ISP, region)
→ If P95 rebuffer ratio > threshold → trigger CDN failover
→ Write to Druid (real-time OLAP for dashboards)
→ Write to Iceberg S3 (historical analysis)

PostgreSQL role:

- CDN configuration management
- ISP peering agreements and quality SLAs
- Historical quality trend reports
- Incident postmortem data

6. Behavioral Questions

Freedom and Responsibility

- "Tell me about a time you made a significant technical decision without asking permission. What was the outcome?"
- "Describe a project you owned completely end-to-end. How did you handle setbacks?"
- "Tell me about a time you delivered bad news about a data issue. How did you communicate it?"

Data-Driven Culture

- "How have you used data to influence a product or engineering decision?"
- "Tell me about a time you were wrong about something you were confident in. What data changed your mind?"
- "Describe how you've measured the success of a system you built."

Scale

- "What's the largest data pipeline you've designed or operated? What was the most challenging aspect?"
- "Tell me about a time you had to redesign a system because it didn't scale."

7. Mock Interview Script

Problem: "Design Netflix's recommendation engine database"

Candidate clarifies:

- "How many users? (247M) How fresh must recommendations be? (< 1 hour staleness acceptable)"
- "One profile per account or multiple? (Multiple profiles per account)"
- "What recommendation types? (Continue watching, Because you watched X, Trending, New releases)"

Design walkthrough:

1. Offline signals: Spark job on Iceberg data (viewing history, ratings, search) → similarity matrices
2. Online signals: Recent views from Redis (< 5 min latency)
3. Re-ranking: Personalization service blends offline + online signals
4. Result storage: Redis cache per user (TTL 30 min) + PostgreSQL for analytics

Key schema: Explained in Task 3 above + the homepage_config table.

Interviewer: "How would you handle the cold start problem for new users?"

Candidate: "For brand new users with no history:

1. Geographic defaults: country-specific trending content
2. Onboarding preferences: prompt users to rate 3 genres
3. Demographic signals: device type, signup source, language
4. Fallback: globally trending content, weighted by recency and review count (Bayesian average)

```
-- New user initial recommendations
SELECT content_id, title, bayesian_avg_rating
FROM content_with_stats
WHERE content_id IN (
    SELECT content_id FROM content_availability
    WHERE country_code = :user_country AND CURRENT_DATE BETWEEN available_from AND
COALESCE(available_to, '9999-12-31')
)
ORDER BY bayesian_avg_rating DESC, view_count DESC
LIMIT 20;
```

8. Evaluation Rubric

L4 (Senior Engineer)
Signal	Pass	Exceeds
SQL	Hard analytical queries	Streaming + batch context
Data Pipeline	Knows Kafka/Flink concepts	Has built production pipelines
Architecture	Service-level designs	Discusses failure modes
Ownership	Demonstrates self-sufficiency	Has driven projects independently

L5 (Staff)
Signal	Pass	Exceeds
SQL	Expert, distributed SQL	Writes novel analytical approaches
Data Platform	Full stack: ingest/store/query	Discusses Iceberg, Flink, Druid
Business Acumen	Understands metrics	Links DB design to business outcomes
Leadership	Has influenced team design	Has driven cross-team platform decisions

9. Red Flags
- No awareness of streaming data patterns (Kafka, Flink, real-time vs. batch)
- Schema designs that can't scale to billions of rows
- No awareness of A/B testing infrastructure
- Cannot discuss trade-offs between Cassandra (NoSQL) and PostgreSQL
- Behavioral stories show dependence on manager for decisions
- No data quality / monitoring considerations
- Doesn't ask about business metrics before designing analytics systems

```



## ## 10. Green Flags

- Mentions Apache Iceberg, Parquet, or Spark in analytics discussions
- Designs with both real-time (Redis/Kafka) and durable (PostgreSQL/Cassandra) layers
- Shows comfort with approximate algorithms (HLL, quantile sketches)
- Links query design to user-facing metrics (watch time, completion rate, churn)
- Mentions Chaos Engineering / fault injection testing for data pipelines
- Behavioral stories show high autonomy and clear ownership
- Can discuss cold start problem for recommendation systems without prompting

---

## ## 11. Salary Bands (Approximate, USD, 2024-2025)

| Level | Base          | Annual Bonus   | RSU (4yr)     | Total Comp     |
|-------|---------------|----------------|---------------|----------------|
| ---   | ---           | ---            | ---           |                |
| L4    | \$170K-\$210K | 0% (RSU-heavy) | \$200K-\$400K | \$220K-\$310K  |
| L5    | \$210K-\$270K | 0%             | \$400K-\$700K | \$310K-\$440K  |
| L6    | \$270K-\$340K | 0%             | \$700K-\$1.4M | \$440K-\$700K  |
| L7    | \$340K-\$420K | 0%             | \$1.4M-\$2.5M | \$700K-\$1.05M |

\*Netflix pays top-of-market base; minimal cash bonus. RSU grants are significant. Los Altos/LA rates. Remote available.\*

---

## ## 12. Preparation Timeline (4-Week Plan)

### ### Week 1: Analytics SQL + Streaming Context

- Day 1-2: Complex analytical SQL: cohort analysis, funnel analysis, A/B testing queries
- Day 3: Study Apache Kafka + Flink fundamentals (Netflix blog posts)
- Day 4: Apache Iceberg format – why it matters for analytics
- Day 5: Study Druid for real-time OLAP (Netflix uses this)
- Day 6-7: Write 10 analytical SQL queries on viewing/content datasets

### ### Week 2: Netflix-Specific Domains

- Day 1: Study Netflix tech blog: recommendation system architecture
- Day 2: A/B testing at scale – Netflix's experimentation platform blog post
- Day 3: Content delivery metrics and quality signals
- Day 4: Churn prediction data signals
- Day 5: Schema design for content catalog (multilingual, multi-region)
- Day 6-7: Mock design interview: design "Continue Watching" feature end-to-end

### ### Week 3: System Design

- Day 1-2: Design personalized homepage (offline + online recommendations)
- Day 3-4: Design streaming quality monitoring pipeline
- Day 4: Cassandra vs. PostgreSQL trade-offs (Netflix uses both)
- Day 5: Freedom & Responsibility culture – read Netflix culture deck
- Day 6-7: Mock interviews with strict 45-min timer

### ### Week 4: Polish

- Day 1-2: Behavioral stories – autonomy, data-driven decisions, ownership
- Day 3: Review Netflix OSS: Conductor (workflow), EVCache, Hystrix
- Day 4-5: Hard analytical SQL (percentiles, regression analysis, cohorts)
- Day 6-7: Rest + light cheat sheet review